Common Core: Setup

Lab L0: Repository, Tools and Flow

Project Structure, File Lists, Make and Git Workflow

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Install and check the open-source tool chain, create your training repository with the directory structure used by every later lab, build the first version of the shared make-based flow (lint, build, simulate, waves), and prove it on a small hello-world block.

Duration	0.5 week (approximately 8–10 hours)
Prerequisites	None
Designs	`counter` (new, hello-world block)
Reused in	Every later lab: all blocks live in this repository and use this flow

Contents

1 Overview **3**

2 Tools **3**

2.1 Tool Chain . . . 3

2.2 Environment Script . . . 4

2.3 Tool Check Script . . . 4

3 Repository Layout **5**

3.1 Top Level . . . 5

3.2 Per-Block Layout . . . 6

3.3 Comparison with Industrial Repositories . . . 6

4 File Lists **7**

5 The Make Flow **7**

5.1 Structure . . . 7

5.2 Targets and Variables . . . 8

5.3 Build Directories and Configurations . . . 9

5.4 Compile-Time Parameters Versus Runtime Plusargs . . . 10

6 Hello World: the `counter` Block **10**
6.1 Specification 10
6.2 Smoke Testbench 11

7 Git Workflow **11**

8 Tasks **12**

9 Deliverables **12**

10 Assessment **12**

11 Review Questions **13**

12 References **13**

1 Overview

A hardware project is more than its RTL. A production block lives in a repository next to its testbench, constraints, lint waivers, documentation and the scripts that build all of them, and dozens of engineers must be able to find, run and reproduce each other's work. Teams that skip this structure pay for it later: results that cannot be reproduced, scripts copied and modified for every block, generated files committed by mistake, and reviews that start with "how do I run this?".

In this lab you set up the repository that you will use for the whole program. Every block you design in Phase 1, verify in Phase 2 and implement in Phase 3 will live here, and every lab will extend the same flow instead of starting a new one. The layout follows large open-source ASIC projects — OpenTitan, Ibex, CVA6 and the PULP `common_cells` library — simplified for training.

By the end of this lab you should be able to:

1. Install the tool chain and verify that every tool has the required version.
2. Explain the purpose of every directory in the training repository and where a new file belongs.
3. Write file lists that work from any directory and include other file lists.
4. Build and run any block with `make`, from its own directory or from the repository root.
5. Distinguish compile-time parameters and defines from runtime plusargs, and keep the build of each configuration separate.
6. Work with the Git workflow of the program: one branch per lab, merge requests, review and completion tags.

2 Tools

2.1 Tool Chain

The labs are written for open-source tools; the team works on the Synopsys cloud, where the industrial equivalents are installed. The flow selects the tool for each step with one make variable, so the targets, file lists and build directories do not change.

Step	Open-source tool	Min. version	Synopsys cloud	Make variable, first lab
Lint	Verilator	5.020	SpyGlass (`sg_shell`), VC SpyGlass (`vc_static_shell`)	`LINT_TOOL`, L0
RTL simulation	Verilator (`--timing`)	5.020	VCS	`SIM`, L0
Waveforms	GTKWave	3.3	Verdi (FSDB)	`WAVES=1`, `make waves`, L0
Build, version control	GNU Make, Git	4.0, 2.30	same	L0
Scripts, plots	Python	3.8	same	`$(PYTHON)`, P3
SV → Verilog	sv2v	0.0.11	not needed	P1
Synthesis	Yosys	0.38	Fusion Compiler (`fc_shell`)	`SYN_TOOL`, P1
Static timing	OpenSTA	2.4	PrimeTime (`pt_shell`)	`STA_TOOL`, P2

The coordinator confirms the versions installed on the team's machines before the program starts

and updates this table. The tools used from P1 on can be installed now or later; `check_tools.sh` reports which are missing.

NOTE | Why versions matter.

Verilator in particular changes quickly: timing support (`--timing`), constrained randomization and coverage support differ between 5.x releases. A testbench that compiles on one version may fail on another. Every log you submit must therefore record the tool versions, and the flow prints them at the start of every run.

2.2 Environment Script

The repository root contains `env.sh` (bash, zsh) and `env.csh` (csh, tcsh, the same variables). Every terminal session starts with `source env.sh` or `source env.csh`; nothing in the flow may depend on the current directory or on absolute paths of one machine. The core of the script:

```
# env.sh -- source this file from the repository root (csh/tcsh: source env.csh)
export REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"   # zsh: ${(%):-%x}
export LIB_DIR="${REPO_ROOT}/lib"
export LIB_STD="${LIB_DIR}/sky130_fd_sc_hd__tt_025C_1v80.lib"   # used from P1
export PATH="${REPO_ROOT}/flow/scripts:${PATH}"
# Tool selection: keep a value that is already set, else pick from the installed tools
: "${SIM:=$(command -v vcs >/dev/null && echo vcs || echo verilator)}"
: "${LINT_TOOL:=verilator}"          # spyglass if Verilator < 5.020 and sg_shell exists
: "${SYN_TOOL:=yosys}"  "${STA_TOOL:=opensta}"   # fc / pt if Yosys / OpenSTA are
   missing
export SIM LINT_TOOL SYN_TOOL STA_TOOL
```

The tool-selection variables only set defaults: every one can be overridden after sourcing or on the make command line (`make sim SIM=verilator`), and `make help` prints the values in use. The script also sets `PYTHON` to the first Python 3.8 or newer it finds (helper scripts call `$(PYTHON)`, never `python3`).

SYNOPSYS CLOUD

On the cloud `env.sh`/`env.csh` contain one clearly marked block of machine-specific locations, each guarded by an existence check (`/global/apps/...` for Verilator, GTKWave, VC Static, SpyGlass, PrimeTime, Fusion Compiler, Formality; VCS and Verdi come from the site setup). Result: `SIM=vcs LINT_TOOL=spyglass SYN_TOOL=fc STA_TOOL=pt`. The installed Verilator is 5.013, older than the 5.020 minimum, and its `--timing` models need g++ $\geq$ 10, so the script points `VERILATOR_CXX` at GCC 12. Design Compiler is installed but not licensed; its generic `class.db` is used only as a bring-up library (`LIB_DB`) until the team PDK is in `lib/`. Makefiles never contain absolute tool paths.

2.3 Tool Check Script

Write `flow/scripts/check_tools.sh`. For every tool in the table it prints the found version, compares it with the minimum and prints `OK`, `TOO OLD` or `MISSING`. It exits with a non-zero code if no tool for a step needed by the current phase is usable (pass the phase as an argument: `check_tools.sh R`, `V`, `P`). Each step is satisfied by its open-source tool or by the Synopsys tool. Output on the team cloud:

```
$ check_tools.sh P
step    open-source                    synopsys                          needed result
make    make 4.2.1>=4.0 OK             -                                 R      OK
git     git 2.46.0>=2.30 OK            -                                 R      OK
python  python3.11 3.11.9>=3.8 OK      -                                 R      OK
lint    verilator 5.013>=5.020 TOO OLD sg_shell 2025.06-SP1 OK           R      OK
lint    -                              vc_static_shell 2024.09-SP2-4 OK  alt    OK
sim     verilator 5.013>=5.020 TOO OLD vcs 2024.09-SP1-1 OK              R      OK
waves   gtkwave 3.3.115>=3.3 OK        verdi 2024.09-SP1-1 OK            R      OK
```

```
conv    sv2v - MISSING              -                              alt    missing (alternative/
    optional)
synth   yosys - MISSING             fc_shell 2025.06-SP1 OK        P      OK
sta     sta - MISSING               pt_shell 2025.06 OK            P      OK
equiv   -                           fm_shell 2026.03-SP2 OK        alt    OK
check_tools: phase P PASS
```

3 Repository Layout

3.1 Top Level

```
asic-onboarding/                    your copy of the training template
+-- README.md                       setup, list of labs and their status
+-- env.sh, env.csh                 source me: REPO_ROOT, LIB_DIR, PATH, tool selection
+-- Makefile                        top level: make <target> IP=<ip>, make check-all
+-- .gitignore                      build/, lib/*.lib, *.vcd, *.fst
+-- doc/
|   +-- reports/weekNN.md           weekly technical summaries
|   +-- reviews/                    notes from review meetings
+-- flow/                           design-independent build infrastructure
|   +-- mk/
|   |   +-- common.mk               paths, configuration tag, file-list expansion   (L0
    )
|   |   +-- lint.mk                 lint                                            (L0
    )
|   |   +-- sim_verilator.mk        build, sim, waves with Verilator (SIM=verilator) (
    L0)
|   |   +-- sim_vcs.mk              build, sim, waves, cov with VCS  (SIM=vcs)       (
    L0)
|   |   +-- regress.mk              regress                                         (V1
    )
|   |   +-- syn.mk                  synth, gls                                      (P1
    )
|   |   +-- sta.mk                  sta                                             (P2
    )
|   +-- syn/  sta/                  generic Yosys and OpenSTA scripts               (P1
    , P2)
|   +-- scripts/                    check_tools.sh, flist_expand.sh, sweep.sh
+-- lib/                            standard-cell libraries (provided, not committed)
+-- hw/
|   +-- ip/<ip>/                    one directory per block
|   +-- top/<top>/                  integration tops (RTL track)
+-- dv/                             shared verification code
|   +-- common/                     testbench utilities                             (V1
    )
|   +-- vip/  uvm/                  verification IP, UVM agents                     (DV
     track)
|   +-- model/                      SV cache model, traces, exploration       (DV, perf
    . tracks)
+-- build/                          ALL generated output -- never committed
    +-- <ip>/<cfg>/{lint,sim,regress,syn,gls,sta}/
```

The guiding rules:

- **Design-specific and design-independent code are separated.** Everything under `flow/` and `dv/common/` works for any block; a block name appears only inside its own directory. Adding a new block never requires a change in `flow/`. (If it does, the flow is not generic yet — fix the flow,

not the block.)

- **Sources and outputs are separated.** Every generated file — simulation models, logs, waveforms, netlists, reports — goes under `build/`. `make clean` deletes it, and `build/` is in `.gitignore`. A fresh clone plus `make` must reproduce every result.
- **Shared resources have one owner.** Shared verification code lives in `dv/`, SystemVerilog models in `dv/model/`, technology files in `lib/`. In a real team these directories have different owners and reviewers (see the `CODEOWNERS` file of CVA6 or OpenTitan).

3.2 Per-Block Layout

Every block uses the same structure. Each phase of the program owns some of the directories, so a finished block shows its whole life cycle.

```
hw/ip/<ip>/
+-- README.md          what the block is, how to run every target, results, limitations
+-- Makefile           block variables + include of the shared flow
+-- doc/
|   +-- spec.md        specification: interface, behavior                    Phase 1
|   +-- design.md      micro-architecture, block and FSM diagrams            Phase 1
|   +-- verification.md  results, bug log                                    Phase 2
|   +-- ppa.md         synthesis and timing results, analysis                Phase 3
+-- rtl/               synthesizable SystemVerilog only, plus <ip>.f        Phase 1
+-- lint/<ip>.vlt      Verilator lint waivers, each one justified           Phase 1
+-- tb/                designer's directed smoke testbench, tb.f             Phase 1
+-- dv/                verification environment                             Phase 2
|   +-- testplan.md    features -> tests, assertions, coverage
|   +-- tb/ env/ tests/ sva/ cov/
|   +-- dv.f
+-- syn/               timing constraints (<ip>.sdc), sweep configurations Phase 3
```

3.3 Comparison with Industrial Repositories

This repository	OpenTitan / Ibex	CVA6 / common_cells
`hw/ip/<ip>/rtl/`	`hw/ip/<ip>/rtl/` (OpenTitan), `rtl/` (Ibex)	`core/` (CVA6), `src/` (common_cells)
`hw/ip/<ip>/dv/`	`hw/ip/<ip>/dv/` (`env`, `tests`, `tb`, `sva`, `cov`)	`verif/` (CVA6), `test/` (common_cells)
`dv/vip/`, `dv/uvm/`	`hw/dv/sv/` (shared agents, `dv_lib`)	`verif/env/`
`dv/testplan.md`	`data/<ip>_testplan.hjson`	`verif/docs/VerifPlans/`
`hw/ip/<ip>/syn/`, `flow/syn/`	`syn/` with Yosys and OpenSTA Tcl (Ibex), `hw/syn/`	`pd/synth/` (CVA6)
`hw/ip/<ip>/lint/`	`lint/` waivers (Ibex, OpenTitan)	`lint/` (common_cells)
`dv/model/`	—	`perf-model/` (CVA6)
`<ip>.f` file lists	FuseSoC `.core` files	`Flist.*` (CVA6), Bender `Bender.yml` (PULP)

Large projects describe their sources with a package manager such as FuseSoC (OpenTitan, Ibex) or Bender (PULP, CVA6), which also resolves dependencies between blocks and generates tool scripts. Plain file lists are the format these tools generate in the end, and they are what you will meet in every commercial flow, so this program uses them directly. Read the FuseSoC and Bender documentation (References) to know what they add.

4 File Lists

A file list (`.f` file) contains the source files of a design in compile order, plus include directories and defines. Simulators, linters and most commercial tools read it with `-f`. The list of sources then lives in exactly one place instead of in every command.

```
// hw/ip/counter/rtl/counter.f
+incdir+${REPO_ROOT}/hw/ip/counter/rtl
${REPO_ROOT}/hw/ip/counter/rtl/counter.sv
```

```
// hw/ip/counter/tb/tb.f
-f ${REPO_ROOT}/hw/ip/counter/rtl/counter.f
${REPO_ROOT}/hw/ip/counter/tb/tb_counter.sv
```

Rules:

- One entry per line; comments start with `//`.
- Packages come before the modules that import them; interfaces before the modules that use them.
- Besides file names, only `+incdir+`, `+define+` and nested `-f` are allowed, so that a small script can expand the list for tools that do not read `-f`.
- Every path starts with `${REPO_ROOT}`. The list then works from any directory, and a block can include another block's list. CVA6 uses the same idea with `${CVA6_REPO_DIR}` in `Flist.ariane`. Verilator expands environment variables in file lists.
- `rtl/<ip>.f` lists only synthesizable files; it is used both for simulation and, from P1 on, for synthesis. Testbench file lists include the RTL list with `-f` and add the testbench files.

Write `flow/scripts/flist_expand.sh`: it reads a file list, follows nested `-f` recursively, substitutes `${REPO_ROOT}`, removes comments and options, drops duplicate entries (the first occurrence wins: from R3 on, two lists may include the same interface file), and prints one absolute file name per line (with `--incdirs` it prints the include directories instead). It is used from P1 on to feed sv2v and Yosys; write and test it now on the `counter` lists.

5 The Make Flow

5.1 Structure

Each block has a small `Makefile` that only sets variables and includes the shared rules. The shared rules are split by tool, so that each phase adds a file instead of editing a large one.

```
# hw/ip/counter/Makefile
IP         := counter
DUT_TOP    := counter
TB_TOP     ?= tb_counter
RTL_FLIST  := rtl/counter.f
TB_FLIST   ?= tb/tb.f
PARAMS     ?= WIDTH=8

include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
```

The top-level `Makefile` delegates to the blocks, so both forms work:

```
make -C hw/ip/counter sim TEST=smoke      # from anywhere
make sim IP=counter TEST=smoke            # from the repository root
make check-all                            # lint + sim (default test) of every block
```

Figure 1 shows what happens on a `make sim`: the top-level `Makefile` only delegates; the block `Makefile` only sets variables; `common.mk` derives the configuration tag and the build directory and includes the simulator rules `sim_$(SIM).mk`; those compile the testbench file list and run the test, and the log ends up in the build directory.

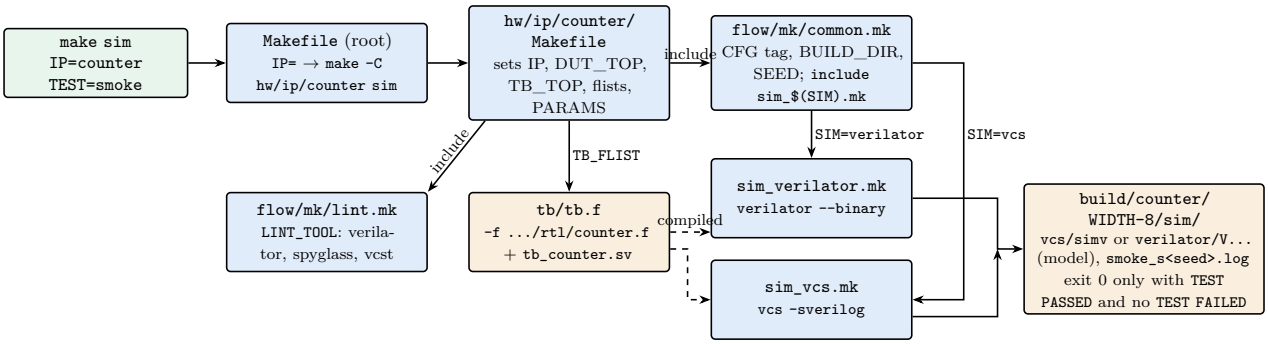


Figure 1: The path of one `make sim`: delegation, block variables, shared rules, the simulator selected with `SIM`, and the outputs under `build/<ip>/<cfg>/sim/`. `make lint` goes the same way through `lint.mk` and `LINT_TOOL`.

`check-all` loops over every directory in `hw/ip/` (and `hw/top/`) that has a `Makefile`. From now on, every merge request must keep `make check-all` passing: a lab never breaks an earlier lab.

TEST TERM | Sanity test (`make check-all`)

The smoke test and lint of every block run together to check that a change did not break anything that used to work. It prints a PASS/FAIL table per block and step (`CHECK_STEPS`, default `lint sim`), keeps the logs in `build/check-all/`, and exits non-zero on any failure. A pass proves only that every block is still alive; it is the minimum before any merge, not a verification.

5.2 Targets and Variables

Target	Function
`help`	Print all targets and all variables with their current values
`lint`	Lint of `LINT_FLIST` with top `LINT_TOP` (defaults: `RTL_FLIST`, `DUT_TOP`) with `LINT_TOOL`: `verilator --lint-only -Wall` with the waiver file `lint/<ip>.vlt` if it exists; `spyglass` / `vcst` on the cloud
`build`	Compile the simulation model of `TB_TOP` (RTL and testbench)
`sim`	Build if needed, run one test, write the log, return a non-zero exit code unless the log contains `TEST PASSED`
`waves`	Open the waveform of the last `WAVES=1` run: GTKWave (FST, Verilator) or Verdi (FSDB, VCS)
`clean`	Delete `build/<ip>/`

Variable	Example	Meaning
PARAMS	WIDTH=16	Compile-time parameter overrides of the testbench top (-G)
DEFINES	MUL_XLEN=8	Compile-time macros (+define+)
TEST	smoke	Test name, passed as +TEST=
SEED	12345	Random seed (used from V1); default: a new seed, printed in the log
PLUSARGS	+NUM_TXN=5000	Extra runtime arguments
WAVES	1	Dump waveforms (FST with Verilator, FSDB with VCS)
SIM	vcs	Simulator: common.mk includes flow/mk/sim_$(SIM).mk (verilator or vcs)
LINT_TOOL	spyglass	Lint tool: verilator, spyglass, vcst
LINT_TOP, LINT_FLIST	arb_mem_lint_top	Lint a wrapper instead of the DUT. Needed from R3 on: Verilator does not accept interface ports on the top-level module, so blocks with interface ports are linted through a small wrapper in lint/

SIM is usable from this lab on: SIM selects sim_$(SIM).mk, so make sim SIM=vcs and make sim SIM=verilator run the same test with the same log name and the same pass/fail rule; a new simulator is one new sim_<tool>.mk, never a change in a block. Later phases add targets (regress in V1, cov in V3, synth and gls in P1, sta in P2, sweep in P3) and variables (COV in V3; UVM=1 in T-DV2, which adds the UVM library to the VCS build and makes TEST the UVM test name; SYN_TOOL, STA_TOOL, CLK_PERIOD in P1/P2; and the dcache-only DV_ENV, which selects one of several testbench environments). Design common.mk so that this is possible without changing what you write now.

SYNOPSYS CLOUD

The same commands with the Synopsys tools: make lint LINT_TOOL=spyglass (SpyGlass lint/lint_rtl goal in sg_shell, fails on errors; LINT_FAIL_ON=warning for warnings too) or LINT_TOOL=vcst (VC SpyGlass); make sim SIM=vcs (VCS, model in build/<ip>/<cfg>/sim/vcs/simv); make sim WAVES=1 then make waves (FSDB in Verdi, no testbench change: the flow generates the dump commands). VCS is four-state, so an uninitialized register shows as X, where Verilator shows 0 or a random value. On the cloud these are the defaults set by env.sh.

5.3 Build Directories and Configurations

All output of a block goes to build/<ip>/<cfg>/<step>/. The configuration tag <cfg> is derived from PARAMS and DEFINES, for example WIDTH-16 or default:

```
# flow/mk/common.mk (excerpt)
ifndef REPO_ROOT
  $(error REPO_ROOT is not set -- run 'source env.sh' in the repository root)
endif

empty :=
space := $(empty) $(empty)
CFG_TAG   := $(subst =,-,$(subst $(space),_,$(strip $(PARAMS) $(DEFINES))))
CFG       := $(or $(CFG_TAG),default)
BUILD_DIR := $(REPO_ROOT)/build/$(IP)/$(CFG)

PARAM_FLAGS  := $(addprefix -G,$(PARAMS))
DEFINE_FLAGS := $(addprefix +define+,$(DEFINES))
```

A change of PARAMS or DEFINES must never reuse a model compiled with other values: that is why the tag is part of the path. Logs are named after the test and seed, for example

`build/counter/WIDTH-16/sim/smoke_s1.log`.

5.4 Compile-Time Parameters Versus Runtime Plusargs

Two different mechanisms configure a simulation. Keep them apart:

	Parameters / defines	**Plusargs**
When fixed	At compile (elaboration) time	At run time
Changes hardware?	Yes: widths, depths, number of flip-flops	No: only testbench behavior
Passed with	`-GDEPTH=8`, `+define+X`	`+NUM_TXN=100` on the run command
Changing it needs	A new compilation	Only a new run
Typical use	`WIDTH`, `DEPTH`, `XLEN`	Test name, seed, number of transactions, verbosity

A design parameter can never be changed by a plusarg, because the hardware has already been built when the simulation starts. The testbench top therefore declares the design parameters itself and passes them to the DUT, so that one `-G` override changes both. The testbench prints its parameters and plusargs at the start of every log.

6 Hello World: the `counter` Block

The first block exists only to prove the repository and the flow. Keep it small; the real design work starts in R1.

6.1 Specification

Port	**Direction**	**Width**	**Description**
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low
`en`	input	1	Count enable
`clr`	input	1	Synchronous clear, has priority over `en`
`count`	output	`WIDTH`	Current value
`wrap`	output	1	One-cycle pulse when `count` goes from `MAX` to 0

Parameters: `WIDTH` (default 8) and `MAX` (default `2**WIDTH-1`). The counter counts from 0 to `MAX` and wraps. Figure 2 shows one implementation.

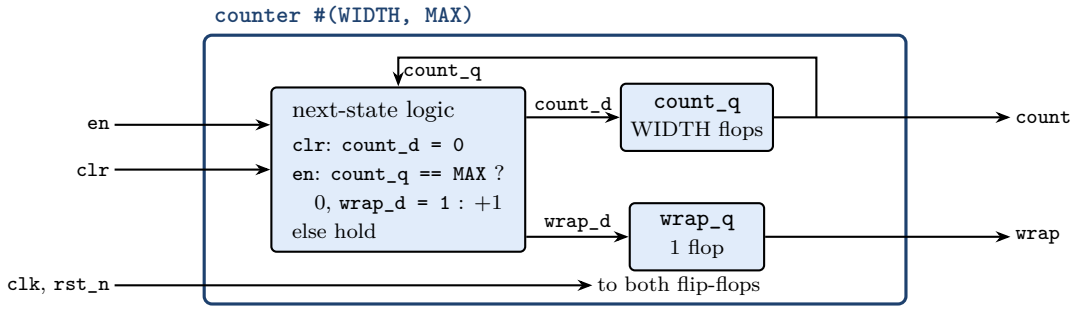


Figure 2: Block diagram of `counter`: one `always_comb` computes the next state (every output has a default, so no latch), one `always_ff` with asynchronous active-low reset holds `count_q` and `wrap_q`. `wrap` is registered: it is high in the cycle after `count` went from `MAX` to 0.

6.2 Smoke Testbench

`tb/tb_counter.sv` is a module with a clock generator, a reset task and a directed test that enables the counter for `2*(MAX+1)+3` cycles, checks every value against an expected value computed in the testbench, checks that `wrap` pulses exactly twice, and checks `clr`. It ends with the standard last line and a watchdog:

```
module tb_counter #(parameter int WIDTH = 8, parameter int MAX = 2**WIDTH-1);
  logic clk = 1'b0, rst_n, en, clr, wrap;
  logic [WIDTH-1:0] count;
  int unsigned n_errors = 0;

  initial forever #5 clk = ~clk;
  counter #(.WIDTH(WIDTH), .MAX(MAX)) dut (.*);

  initial begin
    string test = "smoke";
    void'($value$plusargs("TEST=%s", test));
    $display("CONFIG: TEST=%s WIDTH=%0d MAX=%0d", test, WIDTH, MAX);
    // TODO: reset, directed checks, wrap count, clear
    if (n_errors == 0) $display("TEST PASSED");
    else               $display("TEST FAILED (%0d errors)", n_errors);
    $finish;
  end

  initial begin   // watchdog
    #1ms $display("TEST FAILED (timeout)");
    $finish;
  end
endmodule
```

You will learn in R1 and R2 exactly how a testbench must drive inputs to avoid races with the design; for this lab, change inputs on the falling edge of the clock.

7 Git Workflow

1. Fork (or copy) the training template into your own repository. The coordinator has read access to it.
2. Develop every lab on its own branch, named `lab/<id>-<topic>`, for example `lab/l0-setup`, `lab/r1-seq-mul`.
3. Commit small, self-contained changes with messages that say *what* and *why*: `sync_fifo: fix pointer wrap for DEPTH not a power of two (fails fill_drain, DEPTH=5)`.
4. Open a merge request when the lab is complete. The description lists the deliverables and the commands that reproduce them.
5. The coordinator reviews the code and holds the review meeting (the review questions at the end of each lab). Address the comments with new commits; do not rewrite reviewed history.
6. After approval the branch is merged and tagged `<id>-done` (for example `r1-done`).

Never commit generated files (`build/`, waveforms, netlists, `obj_dir`), large binary files or technology libraries. Small text reports that the documentation refers to may be committed under `hw/ip/<ip>/doc/results/`.

8 Tasks

1. Install the tools, write `check_tools.sh` and `env.sh`, and record the output in the root `README.md`.
2. Create the repository structure of Section 3, with a `README.md` in `flow/`, `dv/` and `dv/model/` that explains in two or three sentences what belongs there.
3. Write `flow/mk/common.mk`, `lint.mk` and `sim_verilator.mk` with the targets and variables of Section 5, and the top-level `Makefile` with `IP=` delegation and `check-all`.
4. Write `flist_expand.sh` and show its output for `hw/ip/counter/tb/tb.f`.
5. Implement the `counter` block and its smoke testbench following the per-block layout, including `doc/spec.md`.
6. Show that these commands work and produce separate build directories:

```
make -C hw/ip/counter lint
make -C hw/ip/counter sim
make -C hw/ip/counter sim PARAMS="WIDTH=4 MAX=9" WAVES=1
make -C hw/ip/counter waves PARAMS="WIDTH=4 MAX=9"
make sim IP=counter TEST=smoke
make check-all
```

7. Break the counter on purpose (for example, let it wrap at `MAX-1`) and show that `make sim` and `make check-all` return a non-zero exit code.

9 Deliverables

Merge request from `lab/l0-setup` containing:

- `env.sh`, `.gitignore`, top-level `Makefile`, root `README.md` with the tool versions.
- `flow/mk/`: `common.mk`, `lint.mk`, `sim_verilator.mk`; `flow/scripts/`: `check_tools.sh`, `flist_expand.sh`.
- `hw/ip/counter/` with `rtl/`, `tb/`, `doc/spec.md`, `Makefile` and `README.md`.
- The empty directory skeleton with its `README.md` files.

10 Assessment

Criterion	Weight
Repository structure follows the specification; nothing generated is committed	25%
Generic flow: targets, variables, configuration-specific build directories	30%
File lists and expansion script	15%
`counter` block and smoke testbench, exit codes	15%
Git workflow and documentation	15%
Total	100%

11 Review Questions

- Where would you put an assertion checker for `sync_fifo`, a UVM agent for `mem_if`, an SDC file for `dcache` and a Python plotting script for cache sweeps? Why there?
- Why must nothing in `flow/` mention a block name?
- Why do file lists use `${REPO_ROOT}` instead of relative paths?
- What goes wrong if `make sim PARAMS="WIDTH=4"` reuses the model compiled for `WIDTH=8`? How does your flow prevent it?
- Why can a plusarg not change `WIDTH`?
- Why does `sim` return a non-zero exit code on failure, and who relies on it?
- Why is `build/` never committed, and what do you commit instead when a result must be documented?
- What do FuseSoC and Bender add compared with plain file lists?
- What would you need to change in `flow/` to add a second block? (Expected answer: nothing.)

12 References

- Verilator User's Guide: `-f`, `-G`, `--timing`, `--trace-fst`, lint warnings and `.vlt` waiver files.
- lowRISC, OpenTitan: `hw/ip/uart/` as an example of a complete block directory.
- lowRISC, Ibex: `rtl/`, `dv/`, `syn/`, `lint/`.
- OpenHW Group, CVA6: `Flist.ariane`, `core/`, `verif/`, `pd/`, `perf-model/`.
- PULP Platform, common_cells and Bender.
- FuseSoC documentation.
- GNU Make manual: variables, pattern rules, `include`, `$(shell)`.
- Pro Git: branching and merging.